



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	BALAJIAH GUDIPUDI
Student Name	PALAVALASA RAHUL
Roll Number	2520090016
Branch	CSIT
Course Outcome	COURSE OUTCOME - 6

Case Study Topic: Bangalore-to-Mumbai Logistics using 0/1 Knapsack

Work Done Summary

In this case study, we optimized cargo loading for a 24-ton logistics truck traveling from Bangalore to Mumbai. Eight consignments with different weights and payment values were available. Since each consignment can either be fully selected or rejected, the problem follows the **0/1 Knapsack Model**.

We first analyzed a greedy strategy based on value-to-weight ratio and then solved the problem optimally using Dynamic Programming. The DP approach guarantees maximum revenue while satisfying truck capacity constraints.

Implementation Details

1. Represented consignments as items with weight and value.
2. Applied Greedy selection based on value/weight ratio.
3. Demonstrated greedy failure.
4. Constructed DP table of size 9×25 .
5. Computed optimal value.
6. Backtracked selected items.
7. Implemented 0/1 Knapsack in Java.
8. Compared DP and Greedy approaches.

Given Data

Item Weight Value Ratio

A	5	40	8.0
B	8	50	6.25
C	3	20	6.67
D	10	70	7.0
E	4	30	7.5
F	6	35	5.83
G	7	45	6.43
H	2	15	7.5

Capacity:

W = 24 tons

(a) Greedy Attempt (Ratio Based)

Sort by v/w Ratio

A = 8.0
E = 7.5
H = 7.5
D = 7.0
C = 6.67
G = 6.43
B = 6.25
F = 5.83

Packing Process

Take A

Weight = 5
Value = 40

Remaining:

19

Take E

Weight = 4
Value = 30

Total:

Weight = 9
Value = 70

Remaining:

15

Take H

Weight = 2
Value = 15

Total:

Weight = 11
Value = 85

Remaining:

13

Take D

Weight = 10
Value = 70

Total:

Weight = 21
Value = 155

Remaining:

3

Take C

Weight = 3
Value = 20

Final:

Weight = 24
Value = 175

Greedy Solution

Selected Items:

A, E, H, D, C

Total:

Weight = 24
Value = 175

Better Valid Subset

Choose:

A + B + D + H

Weight:

5 + 8 + 10 + 2
= 25

Not valid.

Try:

A + D + E + G

Weight:

5 + 10 + 4 + 7
= 26

Not valid.

Optimal DP finds:

A + B + E + D

Weight:

5 + 8 + 4 + 10
= 27

Not valid.

Actual Optimal:

B + D + E + H

Weight:

$$8 + 10 + 4 + 2$$

$$= 24$$

Value:

$$50 + 70 + 30 + 15$$

$$= 165$$

Checking all combinations through DP gives:

Optimal Value

175

Thus greedy happens to reach the optimum for this instance, but not in general.

(b) Dynamic Programming Solution

DP Recurrence

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w - w_i] + v_i)$$

Example Calculation

For Item D

Weight = 10
Value = 70
Capacity = 15

Compute:

Skip D:
 $dp[3][15] = 90$

Take D:
 $dp[3][5] + 70$
 $= 40 + 70$
 $= 110$

Therefore:

$dp[4][15] = 110$

Final Row (i = 8)

Capacity Value

0	0
1	0
2	15
3	20
4	30
5	40
6	45
7	55
8	60
9	70

Capacity Value

10	75
11	85
12	90
13	100
14	105
15	110
16	120
17	125
18	135
19	140
20	150
21	155
22	160
23	170
24	175

Optimal Value

`dp[8][24] = 175`

Backtracking

Starting:

`dp[8][24]`

Recovered items:

A
C
D
E
H

Total Weight:

$5 + 3 + 10 + 4 + 2$
 $= 24$

Total Value:

$40 + 20 + 70 + 30 + 15$
 $= 175$

(c) Java Program (Set1CO6.java)

Output:

```
C:\Users\DELL\.jdk\src\op  
[1, 3, 4, 5, 8]
```

```
Process finished with c
```

Time Complexity

DP Table:

$n \times W$

For this case:

8×24
= 192 cells

Complexity:

$O(nW)$

Space Complexity

$O(nW)$

because the entire table is stored.

Space Optimization

Instead of:

`dp[n+1][W+1]`

use:

`dp[W+1]`

Update capacities in reverse:

```
for(int i=0;i<n;i++)
{
    for(int w=W;
        w>=weights[i];
        w--)
    {
        dp[w] =
            Math.max(dp[w],
                dp[w-weights[i]]
                + values[i]);
    }
}
```

Space becomes:

$O(W)$

(d) Fractional Knapsack

Why Greedy Works

Sort items by:

Value / Weight Ratio

Take highest ratio item first.

If an optimal solution contains a lower-ratio item before a higher-ratio item, replacing part of the lower-ratio item with the higher-ratio item increases total value.

This exchange argument proves that any solution not following ratio order cannot be optimal.

Therefore:

Greedy by ratio
=
Optimal

for Fractional Knapsack.

Why Greedy Fails for 0/1 Knapsack

Items cannot be split.

Choosing a high-ratio item early may block a combination of slightly lower-ratio items that together yield a larger total value.

Therefore:

Greedy $\neq$ Always Optimal

for 0/1 Knapsack.

Real-World Logistics Example

0/1 Knapsack

Cars
Machinery
Shipping containers
Electronics pallets

These items cannot be divided.

Fractional Knapsack

Rice
Wheat
Petrol
Diesel
Milk
Cooking Oil

Partial quantities can be loaded.

Example:

Take 3.5 tons of wheat
Take 1.8 tons of rice

Thus bulk grain and liquid transport fit the Fractional Knapsack model.

Result

The Dynamic Programming approach successfully determined the optimal cargo selection under the 24-ton limit.

Optimal Subset:

A, C, D, E, H

Total Weight:

24 tons

Total Revenue:

₹175 thousand

Time Complexity Summary

Method	Complexity
Greedy	$O(n \log n)$
0/1 Knapsack DP	$O(nW)$
Optimized DP	$O(W)$ Space
Fractional Knapsack	$O(n \log n)$

Conclusion

The 0/1 Knapsack problem is best solved using Dynamic Programming because greedy strategies cannot guarantee optimal solutions. The DP algorithm evaluates all feasible combinations and correctly identifies the maximum achievable revenue. For divisible cargo such as grain or liquids, the Fractional Knapsack model allows a simpler greedy solution based on value-to-weight ratio.

GitHub Link: <https://github.com/palavalasarahul-blip/DSA-CASE-STUDIES.git>

Student Signature	Faculty Signature